

“INTERVIEW PRACTICE PLATFORM”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

RASAGNYA MACHANAVAJHALA

2520090018

NIYATHI KONDUPARTHY

2520090072

VARSHITH COMPLE

2520080016

Under the Esteemed Guidance of

Divya Priya Degala

Assistant Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Declaration

The Project Report entitled **“INTERVIEW PRACTICE PLATFORM”** is a record of bonafide work of **RASAGNYA MACHANAVAJHALA– 2520090018, NIYATHI KONDUPARTHY – 2520090072, VARSHITH COMPLE– 2520080016** submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 10, Team 1, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

RASAGNYA MACHANAVAJHALA – 2520090018

NIYATHI KONDUPARTHY – 2520090072

VARSHITH COMPLE -- 2520080016

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

CERTIFICATE

This is to certify that the project report entitled **“INTERVIEW PRACTICE PLATFORM”** is a bonafide work done and submitted by **RASAGNYA MACHANAVAJHALA– 2520090018, NIYATHI KONDUPARTHY – 2520090072, VARSHITH COMPLE– 2520080016** in partial fulfillment of the requirements of the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT** (25CS1302E), Section 10, Team 1, at KLH University during **2026–2027, Trimester 4.**

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr** , and my guide **Divya Priya Degala**, Department of Computer Science and Information Technology which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Kaja Shareef**, Head of the ***Department for Computer Science and Information Technology*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

RASAGNYA MACHANAVAJHALA – 2520090018

NIYATHI KONDUPARTHY – 2520090072

VARSHITH COMPLE -- 2520080016

Abstract

The Interview Practice Platform – AI Resume Matching & Feedback is an AI-powered application designed to help job seekers identify suitable job opportunities and improve their interview preparation. Many candidates face difficulties due to resume rejection, generic interview questions, and a lack of proper feedback on their interview performance. This project aims to address these challenges by providing a personalized and interactive interview preparation experience.

The proposed system uses Artificial Intelligence (AI) and Natural Language Processing (NLP) to analyze resumes, extract relevant skills and experience, and match candidates with suitable job opportunities. Using spaCy for resume parsing and Sentence Transformers for semantic similarity, the platform ranks job opportunities based on the candidate's profile. It also uses Groq or Gemini AI to generate personalized mock interview questions according to the selected job role and the candidate's skill gaps.

The platform allows candidates to participate in mock interviews using their microphone and camera. Spoken answers are converted into text using faster-whisper, enabling the system to analyze responses and provide AI-based feedback on answer quality, speaking pace, and fillerword usage. Candidates can review their performance, identify areas for improvement, and track their progress through previous interview sessions.

The application follows a three-tier architecture, consisting of a React.js frontend, a Node.js and Express backend, and a Python Flask-based AI microservice. MongoDB Atlas is used to store user details, resumes, job matches, interview sessions, transcripts, and feedback scores. The system uses open-source technologies and free-tier infrastructure to make implementation and deployment more accessible.

Overall, the proposed platform integrates resume analysis, intelligent job matching, personalized mock interviews, and automated performance feedback into a single application. It aims to make interview preparation more accessible, efficient, and personalized while helping candidates improve their skills and build confidence for future interviews.

S.No.	Chapters	Topics	Page No.
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	5
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	7
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 Version Control 3.5 Authentication and Security Libraries	8
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Deployment Architecture	9
5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 Design Decisions and Assumptions	10
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic 6.3 API Integration 6.4 Authentication and Authorization	12
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	13
8	Testing	8.1 Postman API Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	15
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	17

10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	18
S.No.	Chapters	Topics	Page No.
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	19
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	20
13	References	– Documentation, tools, and resources used	21
14	Appendices	– Sample Code Snippets – Installation / Setup Instructions – Default Login Credentials – API Reference Summary – Screenshots (to be added by the student)	22

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Finding the right job and performing well in the interview are two steps that most candidates handle separately and with very little support. On the hiring side, resumes are routinely screened manually or through keyword-based Applicant Tracking Systems (ATS). Industry estimates suggest that more than 75% of resumes are filtered out by ATS or recruiters before a human ever reads them, which means a candidate can be a genuinely good fit for a role and still never be seen for it.

On the preparation side, candidates mostly rely on one-size-fits-all question banks that do not reflect the specific role's likely questions or the candidate's own skill gaps. Practicing alone gives no insight into answer quality, pacing, or delivery, which are the things that actually decide interviews, and role-specific coaching is out of reach for most students on cost grounds.

The Interview Practice Platform – AI Resume Matching & Feedback was built to connect these two steps in a single, free-tier pipeline. The candidate uploads a resume, the platform matches it to suitable roles, generates mock-interview questions tailored to the matched job and the candidate's gaps, captures spoken answers in the browser, and returns structured feedback on both content and delivery.

1.2 Problem Statement

Job seekers struggle to find roles that genuinely fit their skills, and even when they land an interview, most prepare with generic questions that do not reflect the role, or get no feedback on how they actually performed. The problem has three parts:

- No personalized job matching: Resumes are manually screened or keyword-filtered, missing better-fit roles a candidate never sees.
- Generic interview preparation: One-size-fits-all question banks do not target the specific role's likely questions or a candidate's own gaps.
- No feedback loop: Practicing alone gives no insight into answer quality, pacing, or delivery.

1.3 Scope of the Project

The system covers the path from resume to practiced interview for a candidate: uploading a resume (PDF or Word, or pasted text) and extracting skills and experience; ranking matches against a seeded job dataset using semantic similarity; generating role-specific questions from the matched job and the resume gaps; recording spoken answers in the browser; transcribing them and surfacing pacing and filler-word signals; producing LLM-based feedback and scores on content and delivery; and storing sessions, transcripts and scores so progress can be tracked across attempts. The dashboard also offers company-based, coding/technical and HR/behavioral practice modes alongside role-based practice.

The project uses a static seeded job dataset rather than live job boards, and it limits delivery analysis to signals derivable from the transcript and timing. Body-language analysis,

voicemodulation scoring, recruiter-facing views and live job-board integration are treated as future work

1.4 Objectives

- **Accurate resume parsing:** extract skills, experience and role signals from a candidate's resume reliably.
- **Smarter job matching:** rank job fit using semantic similarity rather than keyword filters alone.
- **Role-specific mock interviews:** generate interview questions tailored to the matched job and the candidate's gaps.
- **Actionable feedback:** score spoken answers on content and delivery so candidates know what to improve.
- **Track progress over time:** store sessions, transcripts and scores so candidates can see improvement across attempts.
- **Keep it accessible:** build entirely on free-tier and open-source infrastructure so it stays affordable.

1.5 Importance of the Application

The platform turns a resume, which candidates already have, into a personalised practice plan. Semantic matching surfaces roles that keyword filters would miss, while question generation grounded in the matched job and the candidate's own gaps makes practice relevant rather than generic. Structured feedback on content and delivery closes the feedback loop that is missing when practicing alone, and stored session history lets candidates see whether they are actually improving. Because every component runs on free-tier or open-source tools, the same level of guidance that is normally paid coaching becomes available to any student with a browser and a microphone.

1.6 Target Users / Audience

The primary users are job seekers and students preparing for placements, who use the platform to upload a resume, view ranked role matches, practise mock interviews and review their feedback history on a personal dashboard. The platform is designed around candidate accounts (name, email, college and branch are stored for each user). A recruiter-facing dashboard that exposes candidate readiness and match pipelines is planned as future work.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The application is split into three services, and the AI service loads speech-recognition and embedding models into memory, so the hardware needs are somewhat higher than for a plain CRUD web application. The following specification is recommended for running the full stack on a single development or demonstration machine:

Component	Recommended specification
Processor	Quad-core CPU (dual-core is workable but slower for transcription)
Memory	8 GB RAM (16 GB preferred when running faster-whisper and sentencetransformers locally)
Storage	Around 5 GB free disk space for Python/Node dependencies, spaCy and embedding models, and project source
Peripherals	Working microphone (required) and webcam (for camera capture)
Network	Stable internet connection for MongoDB Atlas and the Groq/Gemini APIs

2.2 Software Requirements

- Node.js (current LTS) with npm, for the Express backend and the React frontend
- Python 3 with pip, for the Flask AI microservice and its libraries (spaCy, sentencetransformers, faster-whisper)
- A spaCy English language model for resume parsing
- A MongoDB Atlas cluster (free tier) for data storage
- A Groq and/or Google Gemini API key for question generation and feedback
- A modern web browser with microphone/camera permission (Chrome, Edge or Firefox)
- Git and GitHub, for version control

2.3 Development Tools and Frameworks

React.js was used to build the candidate-facing frontend; Node.js with Express handles authentication, session orchestration and database access; Python with Flask hosts the AI microservice; spaCy, sentence-transformers and faster-whisper provide parsing, embeddingbased matching and speech-to-text respectively; the Groq/Gemini API supplies the large language model for question generation and feedback; MongoDB Atlas stores users, sessions, transcripts and scores; and Git/GitHub is used for source control.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The frontend is a React.js application with four key areas: a home page that introduces the platform and leads to resume upload; login and registration screens for candidate authentication; forms for resume upload and job preferences that feed the matching engine; and a dashboard that brings match results, mock-interview access and feedback history into one place. Audio and video are captured directly in the browser using the `getUserMedia` and `MediaRecorder` APIs, so no plugins or external applications are needed.

3.2 Backend

The backend is a Node.js and Express service that sits between the React frontend and the Python AI service. It handles authentication and sessions, reads and writes MongoDB, and orchestrates calls to the AI microservice over REST, so that the browser never talks to the AI layer directly.

3.3 Database

MongoDB Atlas is used for persistence. Its flexible, document-based schema suits variablelength transcripts and evolving question sets. Collections reference each other through `ObjectId` values rather than deep nesting, and indexes on candidate ID and session ID support fast dashboard lookups. The data model is described in Chapter 5.

3.4 Version Control

Git and GitHub were used for version control and team collaboration across the three services.

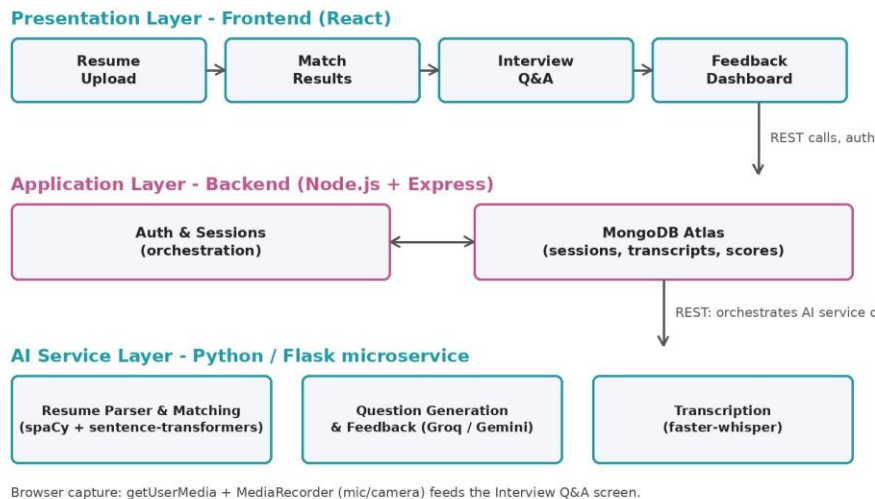
3.5 Authentication and Security Libraries

Authentication and session handling are implemented in the Node.js backend. Passwords are stored only as hashes (the `User` entity holds a `password_hash` field, never a plain-text password), and API keys for the Groq/Gemini services are kept on the server side and are not exposed to the browser.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The platform follows a three-tier architecture with a separate AI microservice: a React frontend, a Node.js/Express backend with MongoDB storage, and a Python/Flask AI service. Figure shows the layers and how a request flows between them.



4.2 Description of Each Layer

The Presentation Layer is the React application. It guides the candidate through resume upload, match results, interview Q&A and the feedback dashboard. It makes REST calls to the Node backend only; authentication is performed via the Node backend and the browser never communicates with MongoDB or the AI service directly.

The Application Layer is the Node.js and Express backend together with MongoDB. It owns authentication and sessions, persists sessions, transcripts and scores in MongoDB, and orchestrates calls to the AI microservice, for example forwarding a resume for parsing or a recorded answer for transcription and then storing the result.

The AI Service Layer is the Python/Flask microservice. It contains the Resume Parser & Matching component (spaCy extraction and sentence-transformers similarity against the seeded job dataset), the Question Generation component (Groq/Gemini, using the matched job and resume gaps) and the Transcription component (faster-whisper). Because it is a separate service reached only over REST, any model or provider in it can be replaced without changing the frontend or backend.

4.3 Deployment Architecture

The three services (React app, Node/Express server and Flask service) run as separate processes that communicate over REST, with MongoDB Atlas as the managed cloud database. All AI components run on free-tier or open-source tools (Groq, faster-whisper, sentence-transformers)

CHAPTER 5: DESIGN

5.1 Database Design Principles

- A flexible, document-based schema suits variable-length transcripts and evolving question sets.
- The core collections are Users, Resumes, Jobs, InterviewSessions and Feedback.
- Collections reference each other via ObjectId rather than deep nesting, which keeps documents small and avoids duplicated data.
- Indexes on candidate ID and session ID make dashboard lookups fast.

5.2 ER Diagram / Database Schema

The database consists of seven related tables, shown in Figure 5.1.

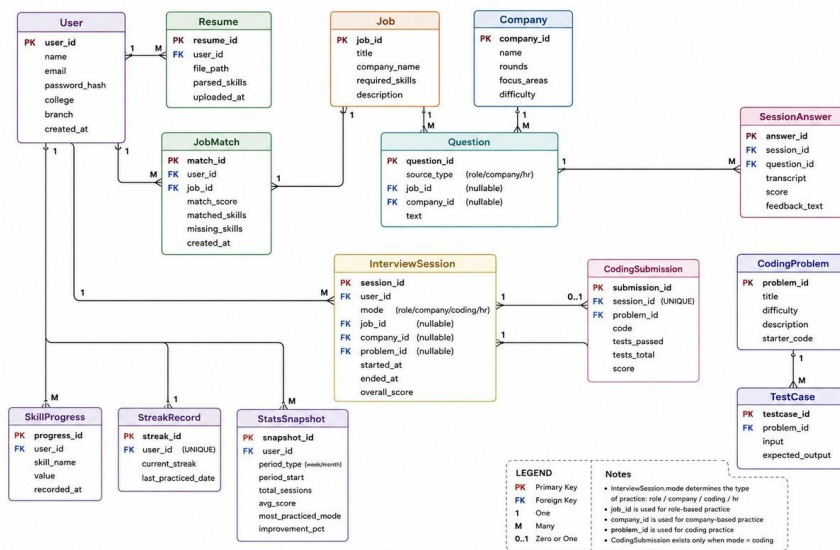


Figure shows the entity-relationship design of the platform. A User uploads Resumes and receives JobMatch records against Jobs (with a match score, matched skills and missing skills). Questions belong to a Job, a Company or the HR set, depending on the practice mode. An InterviewSession records a practice attempt in one of four modes (role, company, coding, HR) and has many SessionAnswer records, each holding a transcript, a score and feedback text. Coding sessions additionally link to a CodingProblem (with TestCases) through a CodingSubmission. SkillProgress, StreakRecord and StatsSnapshot support the progress dashboard.

5.3 Design Decisions and Assumptions

- Node and Python are separate services talking over REST, so the AI logic stays isolated and swappable.

- All AI components use free-tier or open-source tools (Groq, faster-whisper, sentence transformers) to keep the platform affordable.
- MongoDB's flexible schema was chosen because transcripts vary in length and question sets evolve.
- InterviewSession.mode determines the type of practice (role / company / coding / HR); job_id is used for role-based practice, company_id for company-based practice and problem_id for coding practice, and a CodingSubmission exists only when mode = coding.
- Job matching runs against a seeded job dataset instead of live job-board APIs

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The platform is organised into six modules that follow the candidate's journey.

No.	Module	Implementation
1	Resume Upload & Parsing	The candidate uploads a PDF/Word resume (up to 5 MB) or pastes text. The Flask service uses spaCy-based NLP to extract skills and experience from the resume text.
2	Job Matching Engine	Resume and job descriptions are converted to embeddings with sentence-transformers; fit is scored by embedding similarity against the seeded job dataset and the top roles are returned with matched and
3	AI Question Generation	missing skills. An LLM (Groq/Gemini) generates role-specific mock-interview questions from the matched job and the resume gaps.
4	Live Answer Capture	In-browser mic and camera recording with getUserMedia and MediaRecorder; no plugins or external apps required.
5	Speech-to-Text & Analysis	faster-whisper transcribes spoken answers; pacing and filler-word signals are surfaced from the transcript.
6	AI Feedback & Scoring	The LLM returns feedback on content and delivery; scores and comments are stored per session for progress tracking.

The conceptual flow for one practice attempt is: resume → parsed skills → ranked matches → generated questions → recorded answer → transcript → content and delivery feedback → stored scores and dashboard update. A single mock-interview turn chains Node → Flask → LLM (and Whisper for transcription), so asynchronous handling is used to keep the experience responsive.

6.2 Front-end Logic

The React frontend implements four groups of screens: the home page, login/registration, resume-upload and job-preference forms, and the dashboard. After login, the dashboard greets the candidate and shows an interview-readiness score based on recent sessions, a practice streak, four practice modes (Role-Based, Company-Based, Coding / Technical Round and HR / Behavioral Round), a recommendation based on recent score dips, a skill-breakdown chart (content, delivery, confidence, technical, pacing) and a progress panel with weekly/monthly toggle, average score, most-practised mode, improvement percentage, score trend and sessions by mode. Screenshots are given in Appendix E.

6.3 API Integration

The React app communicates only with the Node/Express backend through REST calls. The backend in turn calls the Flask microservice's REST endpoints for parsing and matching, question generation, transcription and feedback, and writes results to MongoDB. Free-tier limits on Groq/Gemini and MongoDB Atlas mean that calls are queued and retried with backoff rather than fired directly when several sessions run at once. A summary of the endpoints is given in Appendix D.

6.4 Authentication and Authorization

Candidates register and log in through the Node backend, which manages sessions; user passwords are stored as hashes. Protected data such as resumes, sessions, transcripts and scores is tied to a user ID so that a candidate sees only their own history. The current demo build shows a login screen noting that any email/password works and also offers a “Continue as guest” option, intended for demonstration.

CHAPTER 7: FEATURES

7.1 List of Main Features

Student-facing features:

- Resume upload by drag-and-drop (PDF or Word, up to 5 MB) or pasted text, with an option to use a sample resume
- Automatic extraction of skills and experience from the resume
- Ranked job matches using embedding-based semantic similarity, with matched and missing skills
- Role-specific mock-interview questions generated from the matched job and resume gaps
- Four practice modes: Role-Based, Company-Based, Coding / Technical Round and HR / Behavioral Round
- In-browser audio/video answer recording with no plugins
- Speech-to-text transcription with pacing and filler-word signals
- LLM-based feedback and scores on content and delivery
- Dashboard with interview-readiness score, practice streak, skill breakdown, score trend and sessions by mode
- Stored sessions, transcripts and scores for progress tracking across attempts

7.2 How Each Feature Works

- **Resume parsing and matching.** The resume text is processed with spaCy to extract skills and experience. Both the resume and each job description are turned into embeddings with sentence-transformers, and similarity is used to rank jobs. The comparison also yields the matched skills and the missing skills, which are the candidate's gaps.
- **Question generation.** The matched job's title, description and required skills, together with the candidate's missing skills, are sent as context to the Groq/Gemini LLM, which returns interview questions aimed at that role and those gaps. Company-based, coding and HR modes draw on company data, coding problems with test cases, and HR question sets respectively.

- **Answer capture and transcription.** The browser records the candidate's answer using `getUserMedia` and `MediaRecorder` and uploads it through the Node backend to the Flask service, where `faster-whisper` converts it to text. Pacing and filler words are derived from the transcript and timing.
- **Feedback and scoring.** The transcript and question are passed to the LLM, which returns structured feedback and scores on content and delivery. The result is saved with the session as a `SessionAnswer` and `Feedback` record.

- **Progress tracking.** Session scores feed the skill-progress, streak and statistics records that drive the dashboard's readiness score, skill breakdown and trend charts.

CHAPTER 8: TESTING

8.1 Postman API Testing

The REST endpoints of the Node backend and of the Flask AI service were exercised with Postman, using collections grouped by area (authentication, resume and matching, questions, transcription, feedback and sessions).

8.2 Integration Testing

Integration testing checked the full chain React → Node → Flask → LLM/Whisper → MongoDB for one complete practice attempt, from resume upload to a stored feedback record visible on the dashboard. Particular attention was paid to the interactions highlighted in Chapter 10: latency across the three services, free-tier rate limits and cross-browser audio/video capture.

8.3 Tools Used for Testing

- Postman – manual and scripted API testing.
- MongoDB Atlas data viewer – verifying stored sessions, transcripts and scores.
- Browser developer tools and manual testing – checking the React frontend and the recording flow in different browsers.

8.4 Test Cases and Results

A representative sample of the test cases executed, drawn from the project's actual test log, is summarized below.

#	Test case	Expected result	Actual result
1	Upload a clean single-column PDF resume	Skills and experience extracted and shown	[Pass / Fail]
2	Upload a multi-column / graphic-heavy resume	Fallback rules produce usable skills (accuracy may drop)	[Pass / Fail]
3	Upload a file larger than 5 MB or an unsupported type	Upload rejected with a clear message	[Pass / Fail]
4	Request job matches for a parsed resume	Ranked list with match score, matched and missing skills	[Pass / Fail]
5	Generate questions for a matched job	Role-specific questions returned and displayed	[Pass / Fail]
6	Record and submit an audio answer	Transcript returned and stored with the session	[Pass / Fail]
7	Submit an answer for feedback	Content and delivery scores and comments returned	[Pass / Fail]
8	Run several simultaneous sessions on free tiers	Requests queued/backed off; no unhandled errors	[Pass / Fail]

9	Use recording in a second browser	Recording works or a clear fallback is shown	[Pass / Fail]

#	Test case	Expected result	Actual result
10	View dashboard after multiple sessions	Readiness, trend and sessions-by-mode reflect stored data	[Pass / Fail]

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

- Create a MongoDB Atlas free-tier cluster and note its connection string.
- Start the Flask AI service: install the Python dependencies, download the spaCy model, set the LLM API key, and run the Flask app.
- Start the Node/Express backend: install dependencies with npm, configure the environment variables, and run the server.
- Start the React frontend: install dependencies with npm and run the development server (or build for production).
- Open the frontend in a browser, grant microphone/camera permission and log in.

9.2 Environment Configuration

Configuration is kept in environment variables rather than in source code, so secrets are never committed to version control. The configuration typically covers: the MongoDB Atlas connection string; the Groq and/or Gemini API key; the URL and port of the Flask AI service; the port of the Node server; and the frontend's allowed origin for CORS. A template file with placeholder values documents the required variables without exposing real keys.

9.3 Hosting of Frontend and Backend

For this submission, both the frontend and backend run as local Node.js processes and the database runs as a local MySQL instance, which is appropriate for development and in-class demonstration. Vite's development proxy forwards frontend API calls to the backend, avoiding any cross-origin configuration during development. A production deployment was not required for this course submission;

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- **Inconsistent resume formats.** spaCy-based parsing works well on clean, single-column resumes, but accuracy drops on tables, multi-column layouts and graphic-heavy templates.
- **Latency across three services.** A single mock-interview turn chains Node → Flask → an LLM call (and Whisper for transcription), which can make a live, in-browser session feel slow.
- **Free-tier rate limits.** Groq/Gemini and MongoDB Atlas free tiers cap request volume and concurrency, which affects testing with several simultaneous mock sessions.
- **Cross-browser audio/video capture.** getUserMedia and MediaRecorder support different codecs and permission flows across browsers, so a recording flow that works in one browser may not behave identically in another.

10.2 Solutions Applied

- For resume formats: extra pre-processing and fallback rules were added rather than relying on parsing alone.

- For latency: careful asynchronous handling across the three services, rather than only trying to speed up individual calls.
- For rate limits: request queuing and graceful backoff instead of firing calls directly.
- For browser differences: explicit fallback handling so the recording flow stays consistent across browsers.

10.3 Current Limitations or Known Bugs

- Job matching runs against a static seeded dataset rather than live job postings.
- Parsing accuracy is still lower on complex resume layouts such as multi-column and graphic-heavy templates.
- Delivery feedback is limited to what can be derived from the transcript (pacing and filler words); body language and voice modulation are not analysed.
- Free-tier API and database limits restrict how many simultaneous sessions can be served.
- Mock interviews and feedback are not yet available in regional languages.
- The demo login accepts any email/password and offers guest access.

CHAPTER 11: CHALLENGES & LIMITATIONS

11.1 Planned Features

- **Body Language Analysis:** MediaPipe-based heuristics for eye contact, posture and fidgeting during mock interviews.
- **Voice Modulation Scoring:** librosa-based pitch, pace and volume analysis for a fuller picture of delivery.
- **Recruiter Dashboard:** a recruiter-facing view into candidate readiness and match pipelines.
- **Live Job Board Integration:** pull real postings via LinkedIn/Indeed APIs instead of a static seed dataset.
- **Multi-Language Interviews:** support mock interviews and feedback in regional languages.
- **Peer Mock Scheduling:** let candidates practice live with peers, with AI scoring both sides.

11.2 Possible Integrations or Optimizations

- Replacing the static seed dataset with live job-board APIs, with embeddings cached so jobs are not re-embedded on every request.
- Streaming transcription and feedback to reduce perceived latency across the Node → Flask → LLM chain.
- A shared request queue with retry/backoff for LLM and database calls to stay within freetier limits.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

This project set out to connect two steps that job seekers normally face separately: finding roles that fit their skills and preparing for the interview with relevant, measurable practice. The Interview Practice Platform parses a candidate's resume with spaCy, ranks roles using sentencetransformers

similarity against a seeded job dataset, generates role-specific questions with a Groq/Gemini LLM, captures spoken answers in the browser, transcribes them with fasterwhisper, and returns structured feedback on content and delivery. Sessions, transcripts and scores are stored in MongoDB so progress can be tracked over time.

The platform follows a three-tier design with a React frontend, a Node.js/Express backend and an isolated Python/Flask AI microservice, and runs entirely on free-tier and open-source infrastructure.

12.2 What Was Achieved

- A personalised pipeline from resume to ranked roles, tailored questions, recorded answers and feedback in one platform.
- Embedding-based matching that goes beyond keyword filtering, with matched and missing skills exposed as the candidate's gaps.
- In-browser recording and transcription with no plugins or external apps.
- A dashboard showing readiness, streak, skill breakdown and progress trends across practice modes.
- A swappable architecture and a low-cost stack that keeps the tool accessible to students.

12.3 Skills Learned

Working on this project gave practical experience in designing a multi-service architecture in which a Node.js backend orchestrates a Python microservice over REST; applying NLP techniques for resume parsing and embedding-based semantic matching; using speech-to-text and LLM APIs within free-tier limits, including queuing and backoff; capturing audio and video in the browser across different browsers; designing a MongoDB data model for sessions, transcripts and scores; building a React dashboard that presents progress data; and collaborating through Git and GitHub.

CHAPTER 13: REFERENCES

1. Wadhwa, Malhotra & Saxena (2020). Automated Resume Screening System using NLP. International Journal of Engineering Research & Technology (IJERT), Vol. 9, Issue 5.
2. Khatri et al. (2025). AI-Powered Automated Resume Screening and Job Matching System Using NLP and ML. IJRESM, Vol. 8, No. 10.
3. JobMatchAI (2026). An Intelligent Job Matching Platform Using Knowledge Graphs, Semantic Search & Explainable AI. arXiv preprint.
4. Radford, A. et al. (2022). Robust Speech Recognition via Large-Scale Weak Supervision (OpenAI Whisper). arXiv:2212.04356.
5. faster-whisper — GitHub repository, <https://github.com/SYSTRAN/faster-whisper>
6. spaCy — Official Documentation, <https://spacy.io>
7. Sentence-Transformers — Documentation, <https://www.sbert.net>
8. Groq API — Documentation, <https://console.groq.com/docs>
9. Google Gemini API — Documentation, <https://ai.google.dev>
10. React — Official Documentation, <https://react.dev>
11. Node.js — Official Documentation, <https://nodejs.org/en/docs>

12. Express.js — Official Documentation, <https://expressjs.com>
13. Flask — Official Documentation, <https://flask.palletsprojects.com>
14. MongoDB Atlas — Documentation, <https://www.mongodb.com/docs/atlas/>
15. MDN Web Docs — MediaStream Recording API and getUserMedia, <https://developer.mozilla.org>
16. Existing tools surveyed: Google Interview Warmup (retired April 2026), Pramp / Exponent Practice, Yoodli.

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippet

Illustrative core of the embedding-based job matching step

```
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer("all-MiniLM-L6-v2") # model name: confirm

def rank_jobs(resume_text, jobs):

    r = model.encode(resume_text, convert_to_tensor=True)

    scored = []
    for job in jobs:

        j = model.encode(job["description"], convert_to_tensor=True)
        scored.append((job["title"], float(util.cos_sim(r, j))))
    return sorted(scored, key=lambda x: x[1], reverse=True)
```

Appendix B: Installation and Setup Instructions

- Clone the repository and create a MongoDB Atlas free-tier cluster.
- AI service: create a Python virtual environment, install requirements, download the spaCy model, set the Groq/Gemini API key and run the Flask app.
- Backend: in the server folder run npm install, configure the environment file and start the server.
- Frontend: in the client folder run npm install and npm run dev.
- Open the frontend URL in a browser and allow microphone/camera access.

Appendix C: Default Login Credentials

The demo build displays the note “This is a demo login — any email/password works” and also provides a “Continue as guest” button

Role	Email	Password
Candidate (demo)	Any email	Any password
Guest	Continue as guest	Not required

Appendix D: API Reference Summary

Method	Endpoint	Purpose
POST	/api/auth/register	Create a candidate account
POST	/api/auth/login	Log in and start a session
POST	/api/resume/upload	Upload a resume; Node forwards it to the AI service for parsing
POST	/api/match	Return ranked job matches with matched / missing skills

POST	/api/interview/questions	Generate role-specific questions for a matched job
POST	/api/interview/answer	Upload a recorded answer for transcription
POST	/api/interview/feedback	Return content and delivery feedback and scores
GET	/api/sessions	List a candidate's past sessions and scores for the dashboard

Appendix E: Screenshots

Frontend screens of the implemented application.

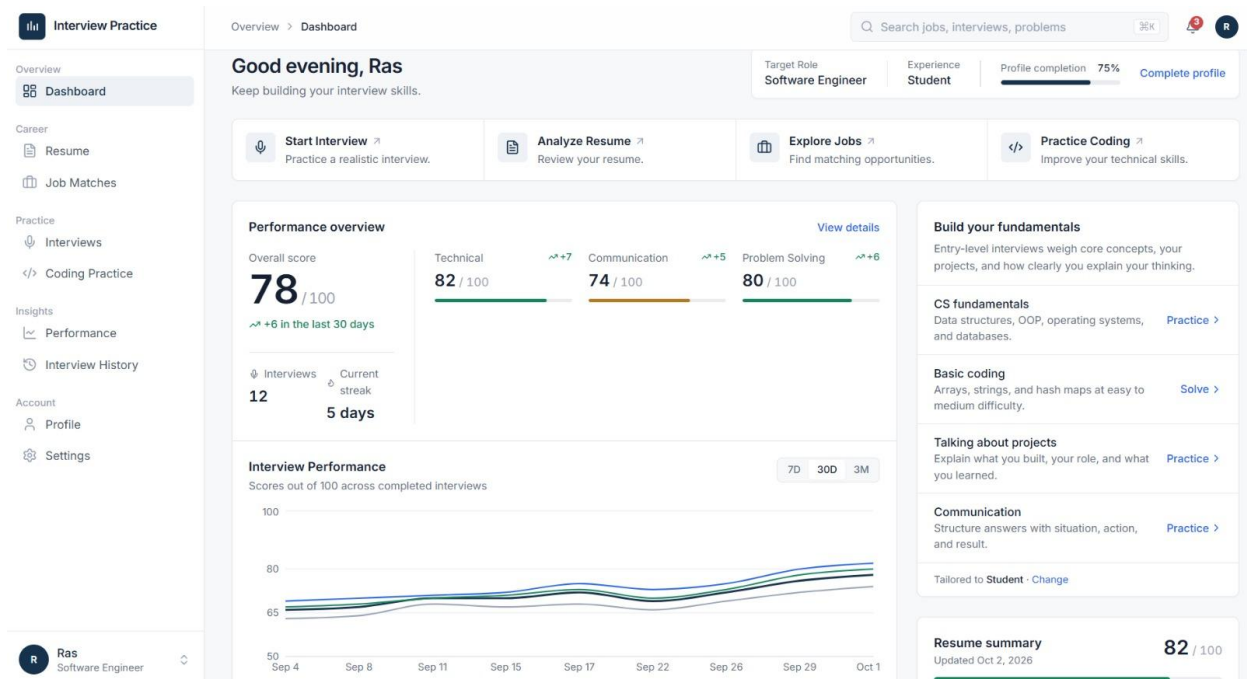


Figure E.1: Login screen

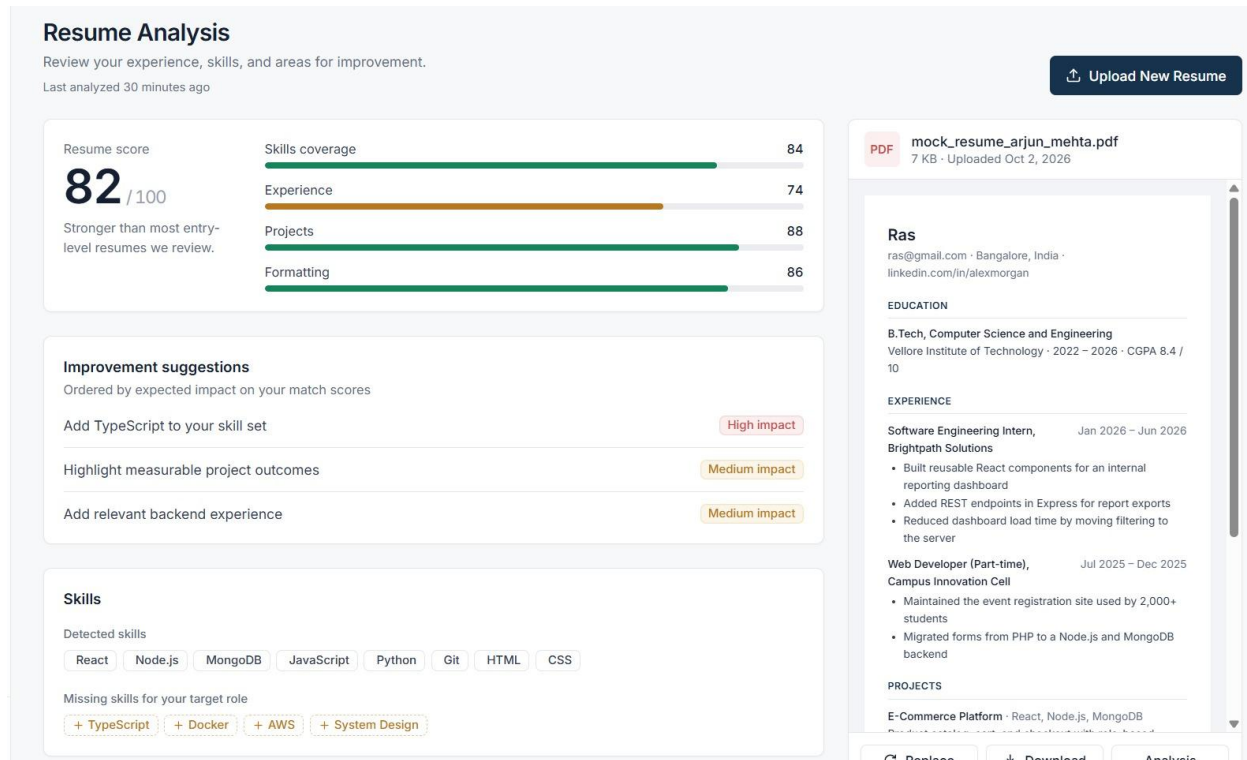


Figure E.2: Resume upload screen

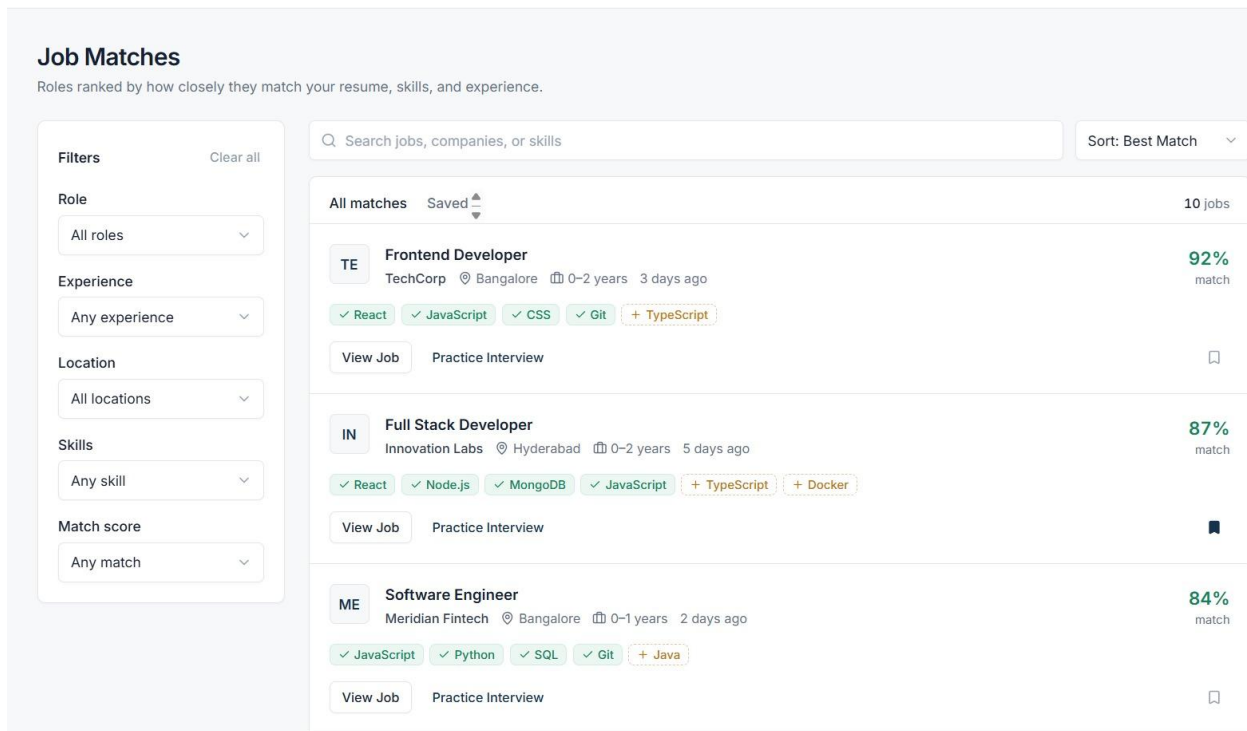


Figure E.3: Candidate dashboard

InterviewIQ — Sample Database Records

Users			
user_id	name	college	branch
1	Niyathi	KL University	IT
2	Rahul Sharma	ABC Engineering	CSE

Jobs			
job_id	title	company_name	score
1	Data Analyst	Microsoft	—
2	Software Engineer	Google	—
3	BI Analyst	Amazon	—

InterviewSessions			
session_id	user_id	mode	score
1	1	role	82.50
2	1	coding	90.00

Questions		
question_id	source_type	question_text
1	role	Explain INNER JOIN vs LEFT J...
2	role	What is data normalization?
3	company	Why do you want to work at M...
4	hr	Tell me about yourself.

Sample records from the InterviewIQ MySQL database

Figure E.4: Sample database records

<pre>_id: ObjectId('6abf81d2b344aefad6d7cad2') slug: "two-sum" __v: 0 createdAt: ISODate('2026-10-02T10:05:06.975+00:00') October 2, 2026 at 10:05:06 AM UTC description: "Return indices of two numbers summing to target." difficulty: "Easy" starterCode: "function twoSum(nums, target) { // TODO }" testCases: Array (2) title: "Two Sum" updatedAt: ISODate('2026-10-02T10:40:19.677+00:00') October 2, 2026 at 10:40:19 AM UTC</pre>	
<pre>_id: ObjectId('6abf81d2b344aefad6d7cad3') slug: "valid-parens" __v: 0 createdAt: ISODate('2026-10-02T10:05:07.007+00:00') October 2, 2026 at 10:05:07 AM UTC description: "Check matching bracket order." difficulty: "Easy" starterCode: "function isValid(s) { // TODO }" testCases: Array (2) title: "Valid Parentheses" updatedAt: ISODate('2026-10-02T10:40:19.711+00:00') October 2, 2026 at 10:40:19 AM UTC</pre>	   
<pre>_id: ObjectId('6abf81d2b344aefad6d7cad4') slug: "longest-substring" __v: 0 createdAt: ISODate('2026-10-02T10:05:07.123+00:00') October 2, 2026 at 10:05:07 AM UTC</pre>	